

Modelos Generativos Profundos

Clase 20: Modelos basados en score (parte II)

Fernando Fêtis Riquelme

Primavera, 2025

Facultad de Ciencias Físicas y Matemáticas

Universidad de Chile

Recuerdo EBM

Score matching

Recuerdo EBM

Modelos basados en energía

Un EBM $p_\theta(x) \propto \exp(-E_\theta(x))$ entrena $E_\theta : \mathbb{R}^D \rightarrow \mathbb{R}$ por máxima verosimilitud (aproximada) usando que:

$$\nabla_\theta \log p_\theta(x) = \mathbb{E}_{p_\theta(x)} [\nabla_\theta E_\theta(x)] - \nabla_\theta E_\theta(x)$$

- $\mathbb{E}_{p_\theta(x)}[\dots]$ se estima usando Monte Carlo, donde las muestras desde $p_\theta(x)$ se generan usando **Langevin sampling**:

$$x_{t+1} = x_t + \frac{\epsilon}{2} \underbrace{\nabla_{x_k} \log p_\theta(x_k)}_{-\nabla_{x_k} E_\theta(x_k)} + \sqrt{\epsilon} z_k, \quad \text{donde } z_k \sim \mathcal{N}(0, I_D).$$

- Los gradientes $\nabla_\theta E_\theta(x)$ y $\nabla_x E_\theta(x)$ se calculan por diferenciación automática de la red neuronal.

Score matching

En la recursión del sampling de Langevin,

$$x_{t+1} = x_t + \frac{\epsilon}{2} \nabla_{x_k} \log p_{\theta}(x_k) + \sqrt{\epsilon} z_k,$$

es necesario computar $\nabla_x \log p_{\theta}(x) \in \mathbb{R}^D$ en cada paso de la iteración, lo que vuelve costoso el entrenamiento y la generación.

Los **modelos basados en score** modelan directamente esta cantidad, denominada **score (de Stein)**, mediante una red neuronal $s_{\theta} : \mathbb{R}^D \rightarrow \mathbb{R}^D$ la cual es entrenada para que $s_{\theta}(x) \approx \nabla_x \log p_{\theta}(x)$.

Esto evita la diferenciación automática en la dinámica de Langevin, volviendo más eficiente el muestreo tanto para entrenamiento como para generación.

Divergencia de Fisher

Aprender directamente el vector $\nabla_x \log p(x)$ es posible ya que una función de score $s(x) = \nabla_x \log p(x)$ define una única función de densidad $p(x)$, y viceversa:

si $\nabla_x \log p_1(x) = \nabla_x \log p_2(x)$ entonces $p_1(x) = p_2(x)$

Para entrenar un modelo basado en score, $s_\theta(x) = \nabla_x \log p_\theta(x)$ se puede utilizar la **divergencia de Fisher**:

$$D_F(p_{\text{data}}(x) \| p_\theta(x)) := \mathbb{E}_{p_{\text{data}}(x)} \left[\frac{1}{2} \|s_\theta(x) - \nabla_x \log p_{\text{data}}(x)\|^2 \right]$$

Score matching

El score real $s_{\text{data}}(x) = \nabla_x \log p_{\text{data}}(x)$ es desconocido, por lo que la divergencia de Fisher no es computable directamente.

Sin embargo, es posible reescribirla de forma tratable, dando paso a la técnica de **score matching**:

$$D_F(p_{\text{data}}(x) \| p_{\theta}(x)) = \mathbb{E}_{p_{\text{data}}(x)} \left[\frac{1}{2} \|s_{\theta}(x)\|^2 + \text{Div}_x(s_{\theta}(x)) \right] + \text{constante}$$

Por lo tanto, es posible entrenar $s_{\theta}(x)$ minimizando la expresión anterior, la cual no depende del score real $s_{\text{data}}(x)$.

Para la generación de muestras, se puede usar Langevin sampling usando directamente $s_{\theta}(x)$ en vez de $\nabla_x \log p_{\theta}(x)$.

Denoising score matching

La descomposición anterior involucra el cálculo de la divergencia $\text{Div}_x(s_\theta(x))$, lo cual puede ser costoso.

Un enfoque alternativo para que $D_F(p_\theta(x) \| p_{\text{data}}(x))$ sea tratable es utilizar otra distribución, $q_\sigma(\tilde{x})$ que sea una versión levemente ruidosa de la distribución real $p_{\text{data}}(x)$.

Eligiendo $q_\sigma(\tilde{x}|x) \sim \mathcal{N}(x, \sigma^2 \mathbf{I}_D)$ (con $\sigma > 0$ un hiperparámetro), su score tiene forma cerrada:

$$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x) = -\frac{\tilde{x} - x}{\sigma^2}$$

Esto es útil para otra posible descomposición de la divergencia de Fisher, dando paso a la técnica de **denoising score matching**.

Denoising score matching

La divergencia de Fisher entre la distribución real alterada $q_\sigma(x)$ y el modelo paramétrico $p_\theta(x)$ se puede reescribir como:

$$\begin{aligned} D_F(q_\sigma(\tilde{x}) \| p_\theta(\tilde{x})) \\ &= \mathbb{E}_{p_{\text{data}}(x), q_\sigma(\tilde{x}|x)} \left[\frac{1}{2} \|s_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x)\|^2 \right] + \text{constante} \\ &= \mathbb{E}_{p_{\text{data}}(x), \epsilon \sim \mathcal{N}(0, I_D)} \left[\frac{1}{2} \left\| s_\theta(x + \sigma\epsilon) + \frac{\epsilon}{\sigma} \right\|^2 \right] + \text{constante} \end{aligned}$$

En consecuencia, se puede entrenar un modelo de score $s_\theta(x)$ para la distribución $q_\sigma(x) \approx p_{\text{data}}(x)$ de forma eficiente.

Notar que el hiperparámetro $\sigma > 0$ produce un trade-off entre el grado de similitud $q_\sigma(x) \approx p_{\text{data}}(x)$ y la varianza de $q_\sigma(\tilde{x}|x)$.

DSM con varios niveles de ruido

El trade-off anterior puede ser evitado al considerar múltiples niveles de ruido en la técnica de DSM.

Si $(\sigma_k)_{k=1}^K \subset \mathbb{R}_{++}$ es una secuencia creciente de ruidos, se pueden considerar distribuciones $q_{\sigma_k}(\tilde{x}|x) \sim \mathcal{N}(x, \sigma_k^2 \mathbf{I}_D)$ para hacer DSM utilizando una única red neuronal $s_\theta : \mathbb{R}^D \times \mathbb{R}_{++} \rightarrow \mathbb{R}^D$ que minimice la función de pérdida combinada

$$\mathcal{L}(\theta) = \sum_{k=1}^K \omega_k \mathbb{E}_{p_{\text{data}}(x), \epsilon \sim \mathcal{N}(0, \mathbf{I}_D)} \left[\frac{1}{2} \left\| s_\theta(x + \sigma_k \epsilon) + \frac{\epsilon}{\sigma_k} \right\|^2 \right],$$

donde $(\omega_k)_{k=1}^K \subset \mathbb{R}_{++}$ son ponderadores asignados a cada nivel de ruido.

Annealed Langevin sampling

Para la generación de muestras desde el modelo anterior se puede generalizar el sampling de Langevin aplicándolo de forma secuencial de forma descendente en los niveles de ruido, y utilizando la muestra generada en el nivel $k+1$ como inicialización para el nivel k . Esto se denomina **annealed Langevin sampling**:

Inicializar $x_{K+1} = 0$

Para $k \in \{K, \dots, 1\}$:

$$\lambda_k = \lambda \left(\frac{\sigma_k}{\sigma_1} \right)^2$$

$$x_{\sigma_k} = \text{Langevin}(\text{score_fn} = s_{\theta}(\cdot, \sigma_k), \text{init} = x_{\sigma_{k+1}}, \text{step} = \lambda_k)$$

En la próxima clase.

- Introducción a los modelos de difusión.

Modelos Generativos Profundos

Clase 20: Modelos basados en score (parte II)